🚀 Traivo Ruttoptimering — Innovationsanalys & Strategisk Roadmap

Dokument: Omfattande analys av nuvarande system + innovativa idéer för världsledande ruttoptimering
Datum: 2026-04-09
Målgrupp: Små/medelstora företag inom paketleveranser, logistik och fältservice

---

1. ANALYS AV NUVARANDE SYSTEM

1.1 Arkitekturöversikt

Traivo är byggt som en **TypeScript-monorepo** med pnpm workspace:

Komponent	Teknologi	Mognad
Backend API	Express 5, Node.js 24	⭐⭐⭐⭐ Solid
Frontend	React 19, Vite 7, Tailwind CSS 4	⭐⭐⭐⭐ Modern
Databas	PostgreSQL + Drizzle ORM	⭐⭐⭐⭐ Välstrukturerad
Optimeringsmotor	ALNS (TypeScript)	⭐⭐⭐ Bra grund
AI-integration	OpenAI GPT-5-mini	⭐⭐⭐ Grundläggande
Kartor/Routing	Leaflet + OSRM	⭐⭐⭐ Funktionellt
Realtid	WebSocket	⭐⭐⭐ Grundläggande
API-design	OpenAPI 3.0 + Orval + Zod	⭐⭐⭐⭐ Professionellt
Förarapp	React Native (Expo)	⭐⭐⭐ Tidig fas
Hållbarhet	CO_2, PM2.5, buller	⭐⭐⭐⭐ Unikt och starkt

1.2 Vad som redan är implementerat och fungerar

✅ Optimeringsmotor (ALNS — 1388 rader)

- **Adaptive Large Neighborhood Search** med simulated annealing
- **4 destroy-operatorer:** Random, Worst, Shaw (spatial clustering), Charging Station removal
- **3 repair-operatorer:** Greedy insertion, Regret-2 insertion, Charging station insertion
- **Multi-objektiv kostnadsfunktion:** Distans (0.45), CO_2 (0.25), ETA-precision (0.15), Förarnöjdhet (0.15)
- **Clarke-Wright savings** för initiallösning

- **k-means klustring** för instanser >100 stopp
- **EV-stöd:** Automatisk laddstationsinsättning med räckviddsberäkning
- **Inkrementell re-optimering** för störningar (behåller stabila rutter)
- **Continuous Optimizer** med progressiv förbättring och WebSocket-uppdateringar
- **Tidsberoende ETA** med hastighetsprofiler per timme (20-50 km/h)

✅ Hållbarhetsmodul (unik styrka)

- CO_2-emissionsberäkning per fordonstyp (diesel, el, hybrid) med well-to-wheel-faktor
- Partikelutsläpp (PM2.5) med broms-/däckslitage och avgasfaktorer
- Bulleranalys med natt-/bostadsområdesdetektering
- Miljöpoäng per rutt (0-100) med CO_2/buller/partiklar-viktning
- Trädequivalenter för CO_2-besparingar

✅ Intelligent Insights Engine

- Realtidsvarningar (sena leveranser, låg batterinivå, kapacitetsöverskridande)
- Prediktiv riskanalys (rushtid, trånga tidsfönster, historisk misslyckanderate per område)
- AI-berikade rekommendationer via GPT-5-mini
- 12+ automatiska insiktstyper

✅ AI Chat Assistant

- Kontextuell chatbot med full flottöversikt
- Streaming SSE-svar
- Proaktiva insikter integrerade i konversationen

✅ Fullständig SaaS-plattform

- Multi-tenant arkitektur med organisationer
- Rollbaserad åtkomst (admin, dispatcher, driver)
- Prenumerationsplaner (starter/pro/enterprise) med användningsgränser
- Shopify-integration för e-handel
- Webhook-system för tredjepartsintegrationer
- Leveransbevis (signatur, foto, streckkod)
- Kundnotifieringar (approaching, delivered, failed, delayed)
- Spårningssida med tracking-koder
- i18n (svenska/engelska)
- Gantt-tidslinje för dispatch

1.3 Styrkor i nuvarande approach

1. **Sofistikerad ALNS-implementation** — Inte bara en grundläggande solver, utan adaptiv operatorviktning, SA-acceptans och multi-objektiv optimering
2. **Hållbarhetsfokus är en genuin differentiator** — Få konkurrenter har PM2.5-beräkning, bulleranalys och miljöpoäng integrerat i optimeringen
3. **Full vertikal integration** — Från optimering till förarapp till kundspårning i ett system
4. **Modern teknikstack** — TypeScript end-to-end, autogenererade API-klienter, Zod-validering
5. **SaaS-redo** — Prenumerationer, multi-tenant, webhook-stöd
6. **Prediktiv insiktsmotor** — Historisk misslyckanderate per adress/område är innovativt

1.4 Svagheter och begränsningar

🔴 Kritiska begränsningar

1. **Distansberäkning med Haversine** — Använder fågelvägsavstånd, inte verkliga vägavstånd. OSRM finns som integration men används INTE i optimeringsloopen. Detta ger 15-30% fel i ruttkostnadsberäkningar.
2. **Trafikprediktion är simulerad, inte reell** — `prediction.ts` genererar "predictions" baserat på hårdkodade hotspots och rushtidsfaktorer med seeded noise. Ingen integration med Google Traffic, HERE, TomTom eller verklig trafikdata.
3. **ETA-profiler är statiska** — Fasta hastighetsprofiler (20-50 km/h) utan hänsyn till fordonstyp, last, väderförhållanden eller vägtyp.
4. **k-means klustring bryter global optimering** — Att dela upp problemet i kluster och optimera varje kluster separat ger suboptimala lösningar. Stopp nära klustergränser tilldelas felaktigt.
5. **Ingen riktig realtidsoptimering** — Continuous Optimizer kör ALNS-iterationer med timer, men reagerar inte på verkliga händelser (trafikolyckor, kundavbokningar, fordonshaverier).
6. **Avståndscache med floating-point precision** — 6-decimalers lat/lon-nycklar kan ge cache-missar vid avrundningsskillnader.

🟡 Viktiga begränsningar

1. **Laddstationsinsättning begränsad till 5 per rutt** — Kan göra långa EV-rutter omöjliga.
2. **Saknar pickup & delivery (PDP)** — Nuvarande modell hanterar bara leveranser, inte hämtning+leveranspar.
3. **Ingen multi-depot support** — Enda depå, vilket begränsar skalbarhet.
4. **Ingen väder-/säsongsintegrering** — Vinterkörning i Sverige kan öka körtider med 30-50%.
5. **Saknar förarskifthantering** — Ingen modellering av pauser, kör-/vilotider (EU-regler), eller skiftbyten.
6. **Ingen batch vs. dynamisk optimering** — Allt hanteras som en enda optimeringskörning.
7. **ALNS i JavaScript/TypeScript** — Signifikant långsammare än C++/Rust för beräkningstunga operationer. Begränsar skalbarhet till ~500-1000 stopp.

🟢 Mindre begränsningar

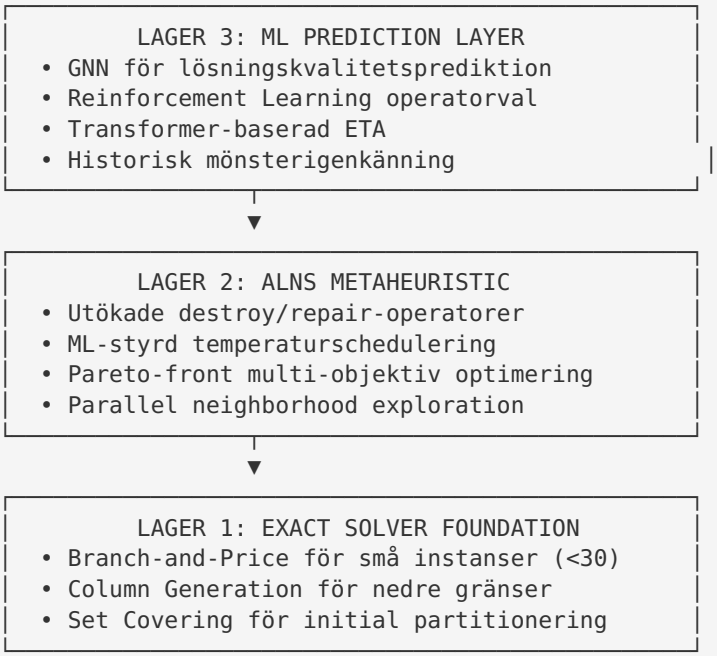
1. **Shaw-removal använder bara spatial närhet** — Bör inkludera temporal och kapacitetssimilaritet.
2. **Regret-2 bör utökas till Regret-k** — Regret-3 eller Regret-k ger bättre resultat.
3. **Ingen or-opt, relocate eller 2-opt förbättringsoperatorer** — Standardoperatorer som saknas.
4. **Saknar stöd för heterogena fordon** — Bara "fossil" och "electric", inte lastbilar, cykelbud, etc.

---

2. INNOVATIVA IDÉER — BORTOM STANDARD VRP

2.1 🧠 Hybrid AI-Optimization Engine (“Traivo Brain”)

Koncept: En tre-lagers optimeringsarkitektur som kombinerar klassisk OR, metaheuristik och maskininlärning.

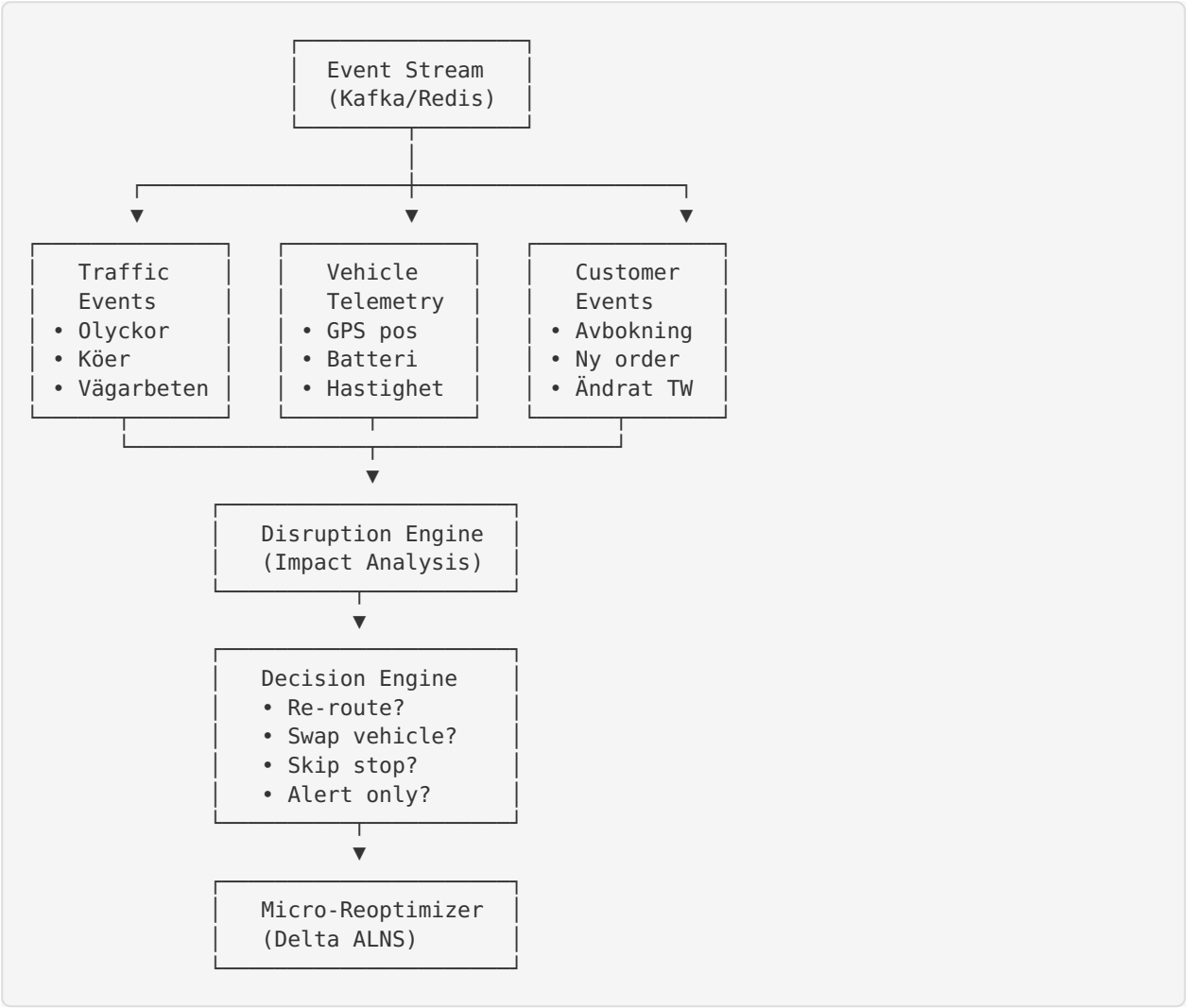


Innovationer:

1. **Graph Neural Network (GNN) för lösningsprediktion:** Träna en GNN på historiska optimeringsresultat. Input: stop-graf med features (position, tidsfönster, vikt). Output: sannolikhet att två stopp ska vara på samma rutt. Använd som “warm start” för ALNS — istället för Clarke-Wright, starta med GNN-predikterad lösning.
2. **Reinforcement Learning för operatorval:** Ersätt den adaptiva viktningen med en RL-agent (DQN/PPO) som lär sig vilken destroy/repair-operator som är bäst givet lösningens nuvarande tillstånd (klustring, belastning, tidsfönster-slack).
3. **Transformer-baserad ETA-prediktion:** Träna en liten transformer (typ 10M parametrar) på historiska leveransdata. Input: position, tid, väderprognos, veckodag, fordonstyp. Output: exakt ETA med konfidensintervall. Ersätter de statiska hastighetsprofilerna.

2.2 🌐 Realtids-Event-Driven Architecture (“Living Routes”)

Koncept: Rutter som lever och anpassar sig i realtid, inte bara omplaneras.



Innovationer:

1. **Disruption Impact Score:** Varje händelse beräknas en "impact score" (0-100) baserat på hur många rutter och stopp som påverkas. Lågpåverkan → justera ETA. Mellannivå → micro-reoptimering. Hög → full reoptimering.
2. **Delta ALNS:** Istället för att optimera om hela flottan, identifiera exakt vilka rutter som påverkas och kör ALNS bara på dem med frozen constraints (stopp som redan levererats kan inte ändras).
3. **Prediktiv disruption:** ML-modell som förutser störningar innan de inträffar. T.ex. "Kl 16:30 på E4 norr om Södertälje har 73% sannolikhet för >15 min fördröjning" baserat på historiska mönster.
4. **Customer-Initiated Rescheduling:** Kunder kan via tracking-sidan begära leverans till annat tidsfönster. Systemet beräknar om det är möjligt utan att påverka andra leveranser.

2.3 🗺️ Intelligent Spatial Engine ("Street-Level Intelligence")

Koncept: Ersätt Haversine-avstånd med riktiga vägavstånd och berika med kontextuell information.

Innovationer:

1. **OSRM i optimeringsloopen:** Implementera en asynkron OSRM-klient med aggressiv cache (LRU med ~100k entries). Batch-hämta avstånd/tider med OSRM Table API istället för point-to-point queries.

2. **Road Segment Intelligence:** Tagga varje vägsegment med metadata: vägtyp (motorväg/lokalväg), hastighetsdata per timme, vikt-/höjdbegränsningar, lastzon-info. Använd i kostnadsfunktionen.

3. **Spatial Temporal Traffic Model:** Dela upp hela köområdet i hexagoner (H3). Varje hexagon har en trafik-hastighetsmatris per timme och veckodag. Baserat på historisk GPS-data från egna fordon.

4. **Parking/Access Intelligence:** ML-modell som lär sig parkeringstider per adress baserat på förares historiska data. Innerstaden = +7 min, industriområde = +1 min. Inkludera i servicetidberäkning.

5. **Turn Penalty Optimization:** Undvik vänstersvängar (à la UPS) genom att lägga till turn penalties i kostnadsberäkningen. Kan spara 3-5% bränsle.

2.4 📊 Prediktiv Demand Intelligence ("Tomorrow's Routes Today")

Koncept: Förutse morgondagens leveransbehov och förbereda optimala rutter proaktivt.

Innovationer:

1. **Demand Forecasting med Prophet/NeuralProphet:** Träna tidsseriemodeller per zon/kundgrupp för att förutse leveransvolymer 1-7 dagar framåt. Dispatchers kan planera fordon och personal proaktivt.

2. **Recurring Pattern Detection:** Identifiera kunder med regelbundna leveransmönster (t.ex. varje tisdag). Föreslå automatisk ruttmall-generering.

3. **Service Duration Learning:** ML-modell som lär sig verklig servicetid per stoppkategori. Input: adresstyp, paketantal, kundhistorik, tid på dygnet, väder. Output: predikterad servicetid med osäkerhetsintervall.

4. **Smart Clustering with Temporal Awareness:** Ersätt k-means med DBSCAN/HDBSCAN som tar hänsyn till både geografi OCH tidsfönster. Stopp i samma område men med inkompatibla tidsfönster bör separeras.

2.5 🌱 Next-Gen Sustainability Engine ("Carbon Zero Routes")

Koncept: Gå från passiv CO_2-rapportering till aktiv miljöoptimering.

Innovationer:

1. **Eco-Routing med topografi:** Integrera höjddata (SRTM/Copernicus) i ruttberäkningen. Undvik branta uppförsbackar för tunga lastbilar. En rutt som är 2 km längre men 200m lägre i höjd kan spara 15% bränsle.

2. **Dynamisk EV-laddstrategi:** Integrera med realtids-elprispriser (redan i DB!) och ladda vid billigaste tidpunkt. Planera laddning vid stopp med lång servicetid. Beräkna optimal SoC (State of Charge) för varje rutt.

3. **Carbon Budget per rutt:** Sätt CO_2-budgetar per rutt/dag/vecka. Optimeringsfunktionen behandlar CO_2 som en hård constraint, inte bara en vikt. "Denna rutt FÅR inte överskrida 50 kg CO_2."

4. **Multi-modal routing:** Kombinera fordon med cykelbud för innerstadsleveranser och lastbilar för ytterstadsrutter. Optimera hela kedjan: lastbil → hub → cykelbud → slutkund.

5. **Scope 3 Emissions Tracking:** Beräkna indirekta utsläpp (packmaterial, lageruppvärmning, retur-logistik) för komplett koldioxidavtrycksrapportering.

6. **Regenerativ bromsningsmodell för EV:** Inkludera energiåtervinning vid nedförsbackar i räck-viddsberäkningen. Baserat på höjddata och fordonsspecifikationer.

2.6 👤 Driver Experience Platform ("Happy Driver = Better Deliveries")

Koncept: Behandla föraren som en kritisk optimeringsresurs, inte bara en begränsning.

Innovationer:

1. **Förarkompetensmodell:** Varje förare har en ML-modell som lär sig deras körstil, servicetidsmön-ster och preferenser. Tilldela rutter baserat på "driver-route fit score."

2. **Gamification Engine:** Poängsystem för eco-driving, on-time delivery, kundbetyg. Leaderboard, achievements, månadsutmaningar. Gamification ökar engagemang med 20-40% i logistikbran-schen.

3. **Fatigue Management:** Integrera med fordonstelemetri (hastighetsvariation, broms-mönster) för att detektera trötthet. Automatisk rast-insättning vid trötthetstecken.

4. **Dynamisk Förartilldelning:** Istället för att fixa fordon → förare-koppling, optimera tilldelningen dagligen baserat på: förarnas kompetens, rutt-komplexitet, kundrelationer, arbetsbelastningsbal-ansering.

5. **Röstguided Navigation med kontext:** Integration med förarappen — röstmeddelanden som "Svårt parkeringsläge vid nästa stopp, förra gången tog parkering 8 minuter" eller "Kund X föredrar att du ringer 5 minuter innan."

2.7 🔗 Intelligent Integration Hub

Koncept: Gör Traivo till navet i kundens logistik-ekosystem.

Innovationer:

1. **Universal Order Ingestion:** Automatisk orderimport från 20+ plattformar (Shopify redan imple-menterat, utöka med WooCommerce, Magento, Klarna, Amazon Seller, PostNord, DHL). Standardiserat format med automatisk geocoding.

2. **ERP/WMS-integration:** Synka lagerstatus. Om en vara tar slut i lagret, ta automatiskt bort stoppet och re-optimera.

3. **Last-Mile Marketplace:** Skapa ett nätverk av SMBs som kan dela överkapacitet. "Företag A har 3 stopp i Södermalm men företag B har en tom bil som redan kör där" → automatisk matchning.

4. **IoT-Sensor Integration:** Temperatur- och fuktövervakning för kylkedjeleveranser. Automatisk re-routing vid temperaturöverskridande.

5. **Return Logistics Automation:** Integrera returflöden i framåtgående rutter. "På väg till stopp 7, hämta en retur vid stopp 6b som ligger 200m bort."

---

3. FÖRESLAGNA AVANCERADE ALGORITMER

3.1 Utökad ALNS — “ALNS+”

```
// Nya destroy-operatorer att implementera:
interface ExtendedDestroyOperators {
  // Relatedness-baserad removal med 4 dimensioner
  relatedRemoval(solution, count, weights: {
    spatial: 0.4,    // Avstånd
    temporal: 0.3,   // Tidsfönster-överlapp
    capacity: 0.2,   // Liknande lastvikt
    historical: 0.1  // Samma rutt i historisk data
  }): ALNSNode[];

  // Ruttbaserad removal – ta bort hela rutter
  routeRemoval(solution, numRoutes): ALNSNode[];

  // Tidsfönster-baserad removal
  timeWindowRemoval(solution, count): ALNSNode[];

  // Proximity-baserad – ta bort stopp nära en slumpmässig punkt
  proximityRemoval(solution, count, radius): ALNSNode[];

  // Historik-driven removal – ta bort stopp som ofta ändras
  historicalRemoval(solution, count, history): ALNSNode[];
}

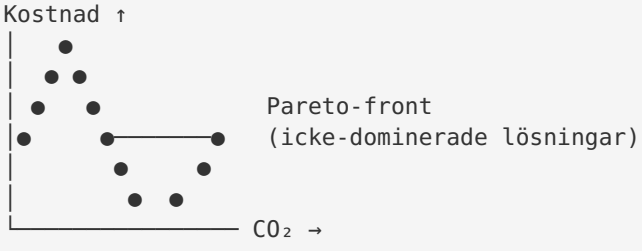
// Nya repair-operatorer:
interface ExtendedRepairOperators {
  // Regret-k med konfigurerbart k
  regretKInsertion(solution, nodes, k: number): void;

  // Deep insertion – pröva att flytta befintliga stopp för att göra plats
  deepInsertion(solution, nodes, maxDepth): void;

  // Parallell insertion med branching
  parallelInsertion(solution, nodes, numThreads): void;
}
```

3.2 Pareto-Front Multi-Objective Optimization

Istället för viktad summa, generera en **Pareto-front** med icke-dominerade lösningar:



Implementation: NSGA-II-inspirerad selektion i ALNS. Behåll en population av lösningar och presentera de 3-5 bästa alternativa rutterna till dispatchers:
- “Snabbaste: 4h 20min, 45 kg CO_2”
- “Grönaste: 4h 55min, 28 kg CO_2”
- “Balanserad: 4h 35min, 35 kg CO_2”

3.3 Branch-and-Price Hybrid

För små instanser (<30 stopp), använd en exakt Branch-and-Price-solver som ger **bevisbart optimala** lösningar. Implementera med:

1. **Master Problem:** Set Partitioning — välj bästa delmängd av rutter
2. **Pricing Problem:** Shortest Path with Resource Constraints (SPPRC) — generera nya kolumner
3. **Branching:** På arc-flow-variabler

Fallback till ALNS för större instanser, men rapportera "optimality gap" baserat på LP-relaxationen.

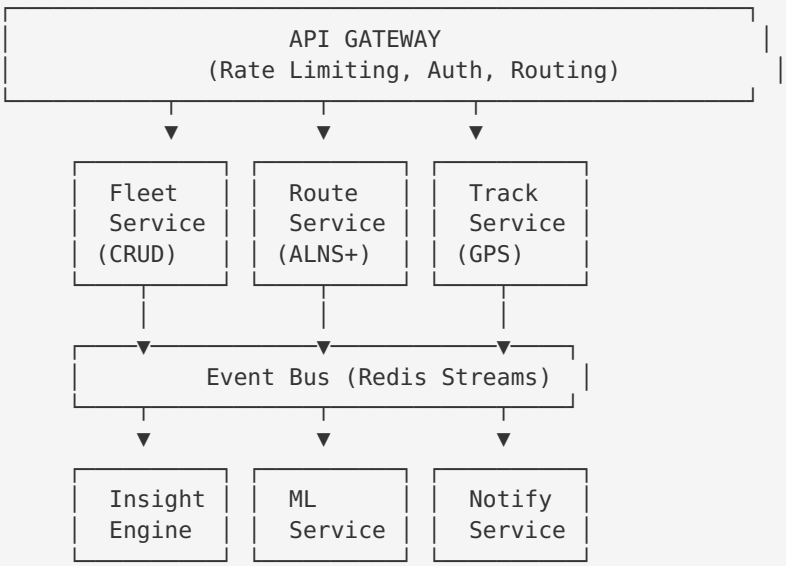
3.4 Konsensus-baserad Klustring

Ersätt k-means med en konsensusmetod:
1. Kör klustring N gånger med olika seeds
2. Beräkna "pair-frequency matrix" — hur ofta hamnar två stopp i samma kluster
3. Använd hierarkisk klustring på pair-frequency-matrisen
4. Tillåt "mjuka" klustergränser — stopp med 40-60% frekvens kan tillhöra båda kluster

---

4. ARKITEKTURFÖRSLAG FÖR SKALBARHET

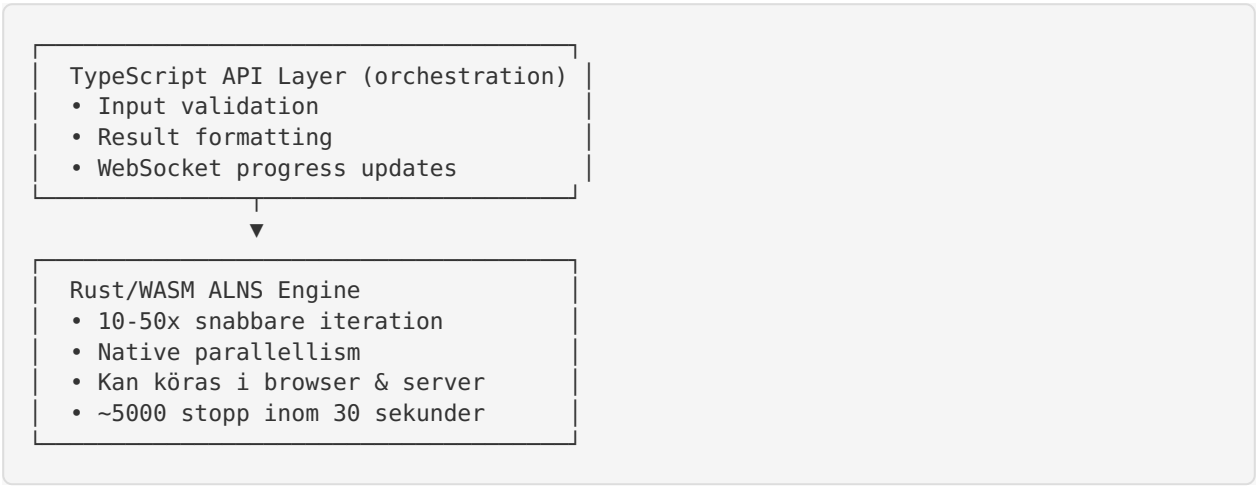
4.1 Mikroservice-evolution



Men — för nuvarande fas, behåll monoliten! Mikrotjänster bara om/när:
- Optimeringsmotorn behöver skalas oberoende
- ML-service behöver GPU-resurser
- GPS-tracking behöver hantera >1000 fordon

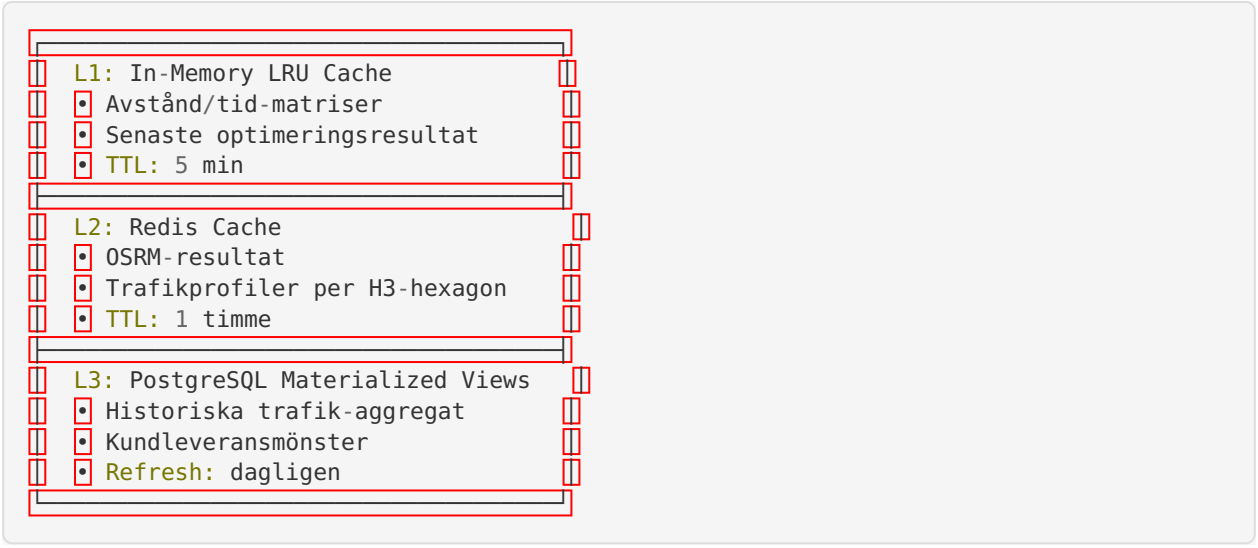
4.2 Optimeringsmotorn i Rust/WASM

Mest kritiska performance-förbättringen: Porta ALNS till Rust och kompilera till WebAssembly.



Alternativ: Använda OR-Tools (Google) via gRPC som en separat service. Ger tillgång till exakt solver + heuristiker.

4.3 Caching & Precomputation Architecture



4.4 Edge Computing för förarappen

Flytta viss optimeringslogik till förarens enhet:
- Lokal re-sekvensering av kommande 3-5 stopp vid trafikstörning
- Offline-kapabel med senast synkade ruttdata
- Camera-baserad adressverifiering (OCR)

---

5. INNOVATIVA FEATURES SOM SKILJER TRAIVO

5.1 “Time Machine” — Simulering & What-If-analys

Dispatchers kan ställa frågor som:
- “Vad händer om vi lägger till 15 stopp i Södermalm?”
- “Hur ser rutterna ut om vi byter ut 2 dieselbilar mot elbilar?”
- “Vad hade hänt om vi startat 1 timme tidigare?”

Implementation: Snabbexekvering av ALNS med modifierade inputs, visualisera skillnad mot nuvarande plan.

5.2 “Fleet DNA” — Organisationsspecifik ML

Varje kund bygger upp sin egen ML-modell baserat på deras data:
- Servicetider per adresstyp (deras historik)
- Trafikmönster i deras körområde
- Förarnas individuella mönster
- Kundbeteenden (hemma/ej hemma-sannolikhet)

Ju längre de använder Traivo, desto bättre blir optimeringen. **Lock-in genom value, inte contract.**

5.3 “Green Score” — Offentlig hållbarhetsrapportering

Generera verifierbara hållbarhetsrapporter:
- Automatisk ESG-rapportering
- Jämförelse med branschgenomsnitt
- “Traivo Green Certificate” som kunder kan visa
- API för att delge CO_2-data till kunders klimatbokslut

5.4 “Smart Zones” — Intelligent geografisk segmentering

Automatisk identifiering av leveranszoner baserat på historisk data:
- Zoner med hög leveransmisslyckanderate → föreslå tidsändringar
- Zoner med konsekvent hög servicetid → justera tid automatiskt
- “Dead zones” utan täckning → identifiera nya affärsmöjligheter

5.5 “Collaborative Routing” — Kundmedverkan

Kunder kan påverka sin leverans:
- Välja föredraget tidsfönster (med dynamic pricing — billigare om flexibelt)
- Ange leveransinstruktioner med foto
- Betygsätta leveransupplevelsen
- Begära förändring i realtid

5.6 “Route Replay” — Lärande från historik

Visuell uppspelning av genomförda rutter med:
- Jämförelse planerat vs faktiskt
- Avvikelsepunkter markerade
- Automatisk identifiering av förbättringsområden
- A/B-testning av olika optimeringsstrategier

5.7 “Autonomous Dispatch” — AI-driven helautomatisk planering

Nivåer av automation:
1. **Assist:** AI föreslår, dispatcher godkänner (nuvarande)
2. **Autopilot:** AI planerar automatiskt, dispatcher övervakar och kan ingripa
3. **Autonomous:** Full automation med undantagshantering — dispatcher hanterar bara eskalationer

---

6. TEKNISK ROADMAP MED PRIORITERINGAR

Fas 1: Grundförbättringar (Vecka 1-4) ⚡ HIGH IMPACT, LOW EFFORT

#	Uppgift	Effort	Impact	Prio
1.1	**OSRM-integration i optimeringsloopen** — Batch Table API med LRU-cache	1 vecka	🔴 Kritisk	P0
1.2	**Utökade ALNS-operatorer** — Relatedness removal, Regret-3, Route removal	1 vecka	🟡 Hög	P0
1.3	**Förbättrad klustring** — DBSCAN med temporal medvetenhet, mjuka gränser	3 dagar	🟡 Hög	P0
1.4	**2-opt/or-opt local search** — Post-optimization efter ALNS	3 dagar	🟡 Hög	P1
1.5	**Shaw removal med fler dimensioner** — Tid, kapacitet, kundtyp	2 dagar	🟢 Medium	P1
1.6	**Fix: distanscache-precision** — Geohash-nycklar istället för float-keys	1 dag	🟢 Medium	P1
1.7	**Fix: EV laddstationsgräns** — Dynamisk gräns baserat på ruttlängd	1 dag	🟢 Medium	P2

Fas 2: Realtid & Trafikdata (Vecka 5-10)

#	Uppgift	Effort	Impact	Prio
2.1	**Verklig trafikdataintegration** — HERE/ TomTom Traffic API	2 veckor	🔴 Kritisk	P0
2.2	**Event-driven disruption engine** — Redis Streams + impact scoring	2 veckor	🟡 Hög	P0
2.3	**Delta ALNS** — Inkrementell reoptimering med frozen constraints	1 vecka	🟡 Hög	P1
2.4	**Live GPS tracking integration** — Fordonsspårning med geofence	2 veckor	🟡 Hög	P1
2.5	**Väderintegration** — SMHI API för körförhållanden	1 vecka	🟢 Medium	P2

Fas 3: ML & Prediktion (Vecka 11-20)

#	Uppgift	Effort	Impact	Prio
3.1	**Service Duration ML-modell** — Lär från historiska leveranser	3 veckor	🟡 Hög	P0
3.2	**ETA Prediction Model** — Transformer/gradient boosting-modell	3 veckor	🟡 Hög	P0
3.3	**Demand Forecasting** — NeuralProphet per zon	2 veckor	🟢 Medium	P1
3.4	**Delivery Success Prediction** — Sannolikhet att kund är hemma	2 veckor	🟢 Medium	P1
3.5	**RL-baserad operatorval i ALNS** — PPO-agent	4 veckor	🟢 Medium	P2
3.6	**GNN warm-start** — Graph Neural Network för initiallösning	4 veckor	🟢 Medium	P2

Fas 4: Skalbarhet & Performance (Vecka 15-25)

#	Uppgift	Effort	Impact	Prio
4.1	**Rust/WASM optimerings-motor** — Porta ALNS-kärnan	6 veckor	🔴 Kritisk	P1
4.2	**Multi-depot support** — Flera startpunkter	2 veckor	🟡 Hög	P1
4.3	**Pickup & De-livery Pairs** — PDP-stöd i ALNS	3 veckor	🟡 Hög	P1
4.4	**Pareto-front multi-objektivt** — NSGA-II-in-spirerad	2 veckor	🟢 Medium	P2
4.5	**Branch-and-Price hybrid** — Exakt solver för små instanser	4 veckor	🟢 Medium	P2

Fas 5: Differentiering & Innovation (Vecka 20-35)

#	Uppgift	Effort	Impact	Prio
5.1	**"Time Machine" simulering** — What-if-analys	2 veckor	🟡 Hög	P1
5.2	**Green Score & ESG-rapportering** — Hållbarhetscertifikat	2 veckor	🟡 Hög	P1
5.3	**Eco-routing med topografi** — Höjddata-integration	3 veckor	🟢 Medium	P1
5.4	**Förarkompetensmodell & gamification**	3 veckor	🟢 Medium	P2
5.5	**Customer self-service portal** — Tidsfönsterval, instruktioner	3 veckor	🟢 Medium	P2
5.6	**Multi-modal routing** — Lastbil + cykelbud	4 veckor	🟢 Medium	P3
5.7	**Last-Mile Marketplace**	6 veckor	🟢 Medium	P3

---

7. KONKRETA NÄSTA STEG (Sprint 1)

Vecka 1 — "Foundation Fix"

Dag 1-2: OSRM i optimeringsloopen

```
1. Implementera OsrmDistanceProvider med batch Table API
2. Skapa asynkron distansmatrix-builder med LRU-cache (50k entries)
3. Ersätt haversineDistance() i evaluateRoute() med osrmDistance()
4. Fallback till Haversine om OSRM är otillgänglig
5. Benchmark: mät optimeringsresultat före/efter
```

Dag 3-4: Nya ALNS-operatorer

```
1. Implementera relatedRemoval med 4-dimensionell relatedness
2. Implementera regret3Insertion och regretKInsertion(k)
3. Implementera routeRemoval (ta bort hela sämsta rutten)
4. Lägg till 2-opt local search som post-processing
5. Uppdatera adaptiv viktning för nya operatorer
```

Dag 5: Förbättrad klustring

```
1. Ersätt k-means med DBSCAN + temporal constraint
2. Implementera konsensus-klustring med N=10 iterationer
3. Tillåt överlapp vid klustergränser
4. Testa med realistiska Stockholm-dataset
```

Vecka 2 — “Smart Distance”

Dag 1-3: Geohash-baserad distanscache

```
1. Byt cache-nycklar till Geohash (precision 8, ~20m)
2. Implementera hierarkisk cache: L1 in-memory, L2 Redis
3. Pre-compute avstånd för vanliga kundpar
4. Invalidera cache baserat på trafikuppdateringar
```

Dag 4-5: Shaw removal förbättring + bugfixes

```
1. Utöka Shaw relatedness med tidsfönster och kapacitet
2. Fix: EV laddstationsgräns dynamisk
3. Fix: Regret-2 infinite regret bias
4. Benchmark hela ALNS-pipelinen
```

---

8. KPIer FÖR FRAMGÅNG

KPI	Nuvarande (uppskattning)	Mål 6 mån	Mål 12 mån
Ruttkvalitet (vs optimal)	~85%	92%	96%
Optimeringstid 100 stopp	~15 sec	5 sec	2 sec
Optimeringstid 500 stopp	~120 sec	30 sec	10 sec
ETA-precision (±min)	±15 min	±8 min	±4 min
CO_2-besparing vs naiv	~15%	25%	35%
Realtids-reaktionstid	N/A	<30 sec	<10 sec
Maximal problemstorlek	~500 stopp	2000 stopp	10000 stopp
Kundnöjdhet (NPS)	N/A	+30	+50

---

9. SAMMANFATTNING

Traivo har en **imponerande grundplattform** med sofistikerad ALNS-optimering, unik hållbarhetsmodul och modern SaaS-arkitektur. De tre mest kritiska förbättringarna för att nå "världsklass" är:

1. 🗺️ **Verkliga vägavstånd** (OSRM i optimeringsloopen) — eliminerar 15-30% felberäkning
2. 🚦 **Verklig trafikdata** (HERE/TomTom) — möjliggör riktig tidsberoende optimering
3. 🧠 **ML-driven ETA & servicetid** — ökar precision från ±15 till ±4 minuter

Det som verkligen kan göra Traivo unikt i marknaden är kombinationen av:
- **Hållbarhetsfokus** (redan starkt, bygg vidare med eco-routing och Green Score)
- **Prediktiv intelligens** ("Fleet DNA" som blir smartare med tiden)
- **SMB-anpassning** (enkel att använda, inte en komplex enterprise-lösning)

Slutord: Bygg inte en "bättre Route4Me" — bygg det system som SMBs vill använda för att det gör dem smartare, grönare och mer lönsamma för varje dag som går.